

7 GÜNLÜK YOĞUNLAŞTIRILMIŞ PROGRAM

Sayısal Kriptografi ve Blockchain

Alt başlıklarıyla ders planı ve
her konu için hazır yapay zeka promptları

38 alt başlık · 110+ hazır prompt · 7 günlük uygulama

Kendi kendine çalışma ve ders anlatımı için tasarlandı



Bu Rehber Nasıl Kullanılır?

Bu doküman bir ansiklopedi değil, bir **çalışma haritasıdır**. Her gün için konular, kısa açıklamalar, anahtar terimler ve en önemlisi **yapay zekaya soracağın hazır promptlar** içerir. Amaç, konuyu buradan okuyup geçmek değil; buradaki açıklamayla zemini kurup, promptlarla derinleşmektir.

1

Günün başında oku

Önce o günün hedefini ve alt başlık açıklamalarını oku. Zihninde bir çerçeve oluşsun; ayrıntı henüz gelmeyecek.

2

Promptları sırayla sor

Her alt başlığın altındaki promptları yapay zekaya sırayla sor. Cevabı okuyup geçme; anlamadığın yerde 'bunu daha basit anlat' diyerek üstüne git.

3

Elle bir örnek çöz

Kriptografi izleyerek değil, hesaplayarak öğrenilir. Her konuda en az bir örneği kâğıt üzerinde ya da kodla kendin çöz.

4

Günün uygulamasını yap

Her günün sonundaki uygulama, o günü kalıcı hâle getiren adımdır. Atlama; kısa tutsan da yap.

5

Kendini sınat

Gün sonundaki sorulara kendi cümlelerinle cevap ver. Takıldığın soru, ertesi günün ilk promptu olsun.

Önemli: Yapay zeka bir öğretmen gibi kullanıldığında güçlüdür, bir kaynak gibi kullanıldığında yanıltıcı olabilir. Özellikle sayısal değerleri, tarihleri ve güncel protokol detaylarını resmî dokümantasyondan doğrula. Son bölümdeki prompt yazma teknikleri, aldığın cevapların kalitesini belirgin biçimde artırır.

Program Akışı

Gün	Konu	Odak	Süre
1	Kriptografinin Temelleri	Güvenliğin dili: kavramlar, matematik ve tehdit modelleri	4-5 saat
2	Simetrik Şifreleme	Tek anahtarla hız: AES, çalışma modları ve anahtar dağıtım problemi	4-5 saat
3	Asimetrik Kriptografi	İki anahtarın gücü: Diffie-Hellman, RSA, eliptik eğriler ve PKI	5-6 saat
4	Hash Fonksiyonları, İmzalar ve Merkle Ağaçları	Blockchain'in yapıştırıcısı: özet fonksiyonları ve kanıtlanabilir bütünlük	4-5 saat
5	Blockchain Mimarisi ve Konsensüs	Dağıtık defterin anatomisi: bloklar, ağ, mutabakat ve cüzdanlar	5-6 saat
6	Akıllı Kontratlar ve Ethereum	Programlanabilir para: EVM, Solidity, token standartları ve DeFi	5-6 saat

Gün	Konu	Odak	Süre
7	Güvenlik, İleri Konular ve Bitirme Projesi	Saldırılar, sıfır bilgi kanıtları, katman-2, post-kuantum ve proje	5-6 saat

GÜN 1 · 4-5 saat

Kriptografinin Temelleri

Güvenliğin dili: kavramlar, matematik ve tehdit modelleri

Günün hedefi: Günün sonunda kriptografinin ne olduğunu, hangi problemleri çözdüğünü ve modern şifrelemenin dayandığı matematiksel zemini anlatabilecek duruma geleceksin.

1.1 Kriptografi Nedir? Kısa Tarihçe

Kriptografi, bir mesajı yalnızca yetkili tarafların okuyabileceği hâle getirme bilimidir. Sezar şifresi (harfleri kaydırma) ve Vigenère (anahtar kelimeyle kaydırma) klasik örneklerdir; II. Dünya Savaşı'nda Enigma makinesi mekanik kriptografinin zirvesiydi. 1976'da Diffie-Hellman makalesiyle 'açık anahtar' fikri doğdu ve kriptografi askerî bir sırdan herkesin kullandığı bir altyapıya dönüştü. Blockchain, bu birikimin üzerine kurulmuştur.

Anahtar terimler: düz metin (plaintext), şifreli metin (ciphertext), anahtar, şifreleme/deşifreleme, kriptanaliz

► YAPAY ZEKAYA SOR

1. “Kriptografinin tarihini Sezar şifresinden modern açık anahtarlı sisteme kadar 5 dönüm noktasıyla anlat; her dönüm noktasında hangi problem çözüldü, hangi yeni problem doğdu?”
2. “Sezar ve Vigenère şifrelerini bana adım adım örnekle anlat, sonra ikisinin de neden bugün güvensiz olduğunu frekans analizi üzerinden göster.”
3. “Enigma nasıl çalışıyordu ve nasıl kırıldı? Bunu 'anahtar uzayı büyük olmasına rağmen neden kırıldı' sorusuna cevap verecek şekilde açıkla.”

1.2 Bilgi Güvenliğinin Beş Sütunu

Her güvenlik sistemi şu hedeflere hizmet eder: Gizlilik (confidentiality) mesajı başkası okuyamaz; Bütünlük (integrity) mesaj değiştirilmemiştir; Kimlik doğrulama (authentication) karşı taraf iddia ettiği kişidir; İnkâr edilemezlik (non-repudiation) gönderen 'ben göndermedim' diyemez; Erişilebilirlik (availability) sistem ihtiyaç anında çalışır. Şifreleme gizliliği, hash bütünlüğü, dijital imza ise kimlik doğrulama ve inkâr edilemezliği sağlar. Blockchain'i anlamamanın anahtarı, hangi aracın hangi sütunu taşıdığını bilmektir.

Anahtar terimler: CIA üçlüsü, non-repudiation, authentication, authorization

► YAPAY ZEKAYA SOR

1. “Gizlilik, bütünlük, kimlik doğrulama, inkâr edilemezlik ve erişilebilirlik kavramlarını bir banka havalesi senaryosu üzerinden tek tek örnekle anlat.”
2. “Şifreleme, hash ve dijital imza araçlarının her birinin hangi güvenlik hedefini sağladığını, hangisini SAĞLAMADIĞINI bir tablo hâlinde göster.”
3. “Bana bu beş kavramı karıştırmam için tasarlanmış 5 soruluk bir çoktan seçmeli test hazırla ve cevaplarımı gerekçeleriyle değerlendir.”

1.3 Modüler Aritmetik ve Sayılar Teorisi

Modern kriptografi 'saat aritmetiği' üzerine kuruludur: $17 \bmod 12 = 5$. Bu yapıda toplama, çarpma ve üs alma tanımlıdır ama işlemler tek yönlü davranır; $a^b \bmod n$ hesaplamak kolay, tersini bulmak (ayrık logaritma) pratikte imkânsızdır. Asal sayılar, Euler phi fonksiyonu, EBOB (Öklid algoritması) ve modüler çarpma göre ters eleman RSA'nın; sonlu cisimler ise AES ve eliptik eğrilerin temelidir. Bu bölüm zor gelebilir ama kriptografiyi ezberden kurtaran tek şeydir.

Anahtar terimler: mod, asal sayı, Öklid algoritması, Euler phi, modüler ters, ayrık logaritma, sonlu cisim

► YAPAY ZEKAYA SOR

1. “Modüler aritmetiği hiç matematik bilmeyen birine anlatır gibi anlat, sonra $7^{11} \bmod 13$ işlemini hızlı üs alma yöntemiyle adım adım çöz.”
2. “Genişletilmiş Öklid algoritmasıyla 17'nin mod 43'teki çarpma göre tersini bul; her satırda ne yaptığını açıklayarak ilerle.”
3. “‘Tek yönlü fonksiyon’ ve ‘kapaklı tek yönlü fonksiyon’ (trapdoor) kavramlarını günlük hayattan benzetmelerle açıkla ve kriptografideki üç örneğini ver.”

1.4 Entropi, Rastgelelik ve Anahtar Uzayı

Bir sistemin gücü anahtarın tahmin edilemezliğinden gelir. 128 bitlik bir anahtarın 2^{128} olasılığı vardır; kaba kuvvetle denemek evrenin yaşından uzun sürer. Ancak anahtar zayıf bir rastgelelikten üretilirse (öngörülebilir tohum, kötü PRNG) uzunluk hiçbir şey ifade etmez. Bu yüzden kriptografide 'kriptografik olarak güvenli rastgele sayı üretici' (CSPRNG) kullanılır. Blockchain cüzdanlarının çalınmasının en yaygın sebeplerinden biri zayıf rastgelelikle üretilmiş özel anahtarlardır.

Anahtar terimler: entropi, CSPRNG, tohum (seed), kaba kuvvet, anahtar uzayı, doğum günü paradoksu

► YAPAY ZEKAYA SOR

1. “Entropiyi bit cinsinden açıkla: 8 karakterlik bir parolanın entropisi nedir, 12 kelimelik bir seed phrase'in entropisi nedir, karşılaştır.”
2. “Rastgele sayı üreticinin zayıf olması yüzünden gerçekten para kaybedilen kriptografi vakalarını anlat ve teknik sebebini açıkla.”
3. “128 bit ve 256 bit anahtarları kaba kuvvetle kırmanın maliyetini enerji ve süre cinsinden hesapla; varsayımlarını açıkça yaz.”

1.5 Kerckhoffs Prensibi ve Tehdit Modelleri

Kerckhoffs prensibi: Bir sistemin güvenliği algoritmanın gizliliğine değil, yalnızca anahtarın gizliliğine dayanmalıdır. 'Gizlilikle güvenlik' (security through obscurity) bir strateji değildir; bu yüzden AES, SHA-256 ve Bitcoin'in kodu herkese açıktır. Tehdit modeli ise 'saldırgan kim, neyi görebiliyor, ne yapabiliyor' sorusunun cevabıdır: pasif dinleyici mi, araya giren (MITM) mi, sistemin içindeki kötü niyetli katılımcı mı? Doğru tehdit modeli olmadan hiçbir güvenlik iddiası anlamlı değildir.

Anahtar terimler: Kerckhoffs prensibi, security through obscurity, tehdit modeli, MITM, pasif/aktif saldırı

► YAPAY ZEKAYA SOR

1. “Kerckhoffs prensibini savun ve ona karşı çıkan argümanları da sun; askerî sistemlerde neden bazen ihlal edildiğini tartış.”
2. “Bir kripto para cüzdanı uygulaması için tehdit modeli çıkar: varlıklar, saldırganlar, saldırı yüzeyleri ve karşı önlemler tablosu hazırla.”
3. “Ortakdaki adam (MITM) saldırısını bir mesajlaşma senaryosunda canlandır ve hangi kriptografik aracın bunu engellediğini göster.”

■ GÜNÜN UYGULAMASI

Python ile Sezar ve Vigenère şifresini yaz, ardından frekans analiziyle kendi şifreni kır. Sonra 'secrets' modülüyle 256 bitlik güvenli bir anahtar üret ve `os.urandom` ile `random.random()` farkını gözlemle.

● KENDİNİ SINA

1. Şifreleme hangi güvenlik hedefini tek başına SAĞLAMAZ?
2. Ayrık logaritma problemi neden kriptografide değerlidir?
3. Kerckhoffs prensibine göre Bitcoin'in kodunun açık olması bir zafiyet midir? Neden?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 2 · 4-5 saat

Simetrik Şifreleme

Tek anahtarla hız: AES, çalışma modları ve anahtar dağıtım problemi

Günün hedefi: AES'in nasıl çalıştığını, hangi modda hangi riskin olduğunu ve simetrik şifrelemenin neden tek başına yetmediğini açıklayabilecek duruma geleceksin.

2.1 Simetrik Şifrelemenin Mantığı

Simetrik şifrelemede şifreleyen ve çözen taraf aynı anahtarı kullanır. Çok hızlıdır: modern işlemciler AES'i donanımda saniyede gigabaytlarca veri için çalıştırır. Bu yüzden disk şifreleme, HTTPS trafiğinin gövdesi ve veritabanı şifrelemesi hep simetriktir. Zayıf noktası tek bir sorudur: aynı anahtarı karşı tarafa nasıl güvenle ulaştıracaksın? n kişilik bir ağda $n(n-1)/2$ farklı anahtar gerekir; 1000 kişi için yaklaşık 500 bin anahtar demektir.

Anahtar terimler: simetrik anahtar, blok şifre, akış şifresi, XOR, one-time pad

► YAPAY ZEKAYA SOR

1. “Simetrik ve asimetrik şifrelemeyi hız, anahtar sayısı, kullanım alanı ve güvenlik varsayımı açısından karşılaştıran bir tablo hazırla.”
2. “One-time pad'i anlat: neden matematiksel olarak kırılmaz ve neden pratikte kullanılamaz?”
3. “Blok şifre ile akış şifresi arasındaki farkı örneklerle (AES vs ChaCha20) anlat; hangi durumda hangisi tercih edilir?”

2.2 DES'ten AES'e: Feistel ve SPN Yapıları

DES (1977) 56 bitlik anahtarıyla Feistel yapısını kullanıyordu: veri iki yarıya bölünür, turlarla karıştırılır ve aynı devre hem şifreler hem çözer. 1990'larda 56 bit kaba kuvvete yenik düşünce NIST bir yarışma açtı ve 2001'de Rijndael algoritması AES olarak seçildi. AES bir SPN (Substitution-Permutation Network) yapısıdır: değiştirme (confusion) ve yayma (diffusion) katmanlarını tur tur uygular. Shannon'ın confusion/diffusion ilkesi tüm modern blok şifrelerin omurgasıdır.

Anahtar terimler: DES, 3DES, Feistel ağı, SPN, confusion, diffusion, tur (round), S-box

► YAPAY ZEKAYA SOR

1. “Feistel yapısını çiz gibi anlat: neden şifreleme ve çözme aynı devreyle yapılabilir?”
2. “Shannon'ın 'confusion' ve 'diffusion' ilkelerini AES'in adımları üzerinde tek tek eşleştirerek göster.”
3. “DES neden emekliye ayrıldı, 3DES neden geçici bir çözümdü ve AES yarışması nasıl yürütüldü? Kronolojik anlat.”

2.3 AES'in İç Yapısı

AES 128 bitlik blokları 4x4'lük bir bayt matrisinde işler ve her turda dört adım uygular: SubBytes (her baytı S-box tablosuyla değiştirir, doğrusal olmayanlığı sağlar), ShiftRows (satırları kaydırır), MixColumns (sütunları sonlu cisimde çarpar, yaymayı sağlar) ve AddRoundKey (tur anahtarıyla XOR'lar). Anahtar uzunluğuna göre 10, 12 veya 14 tur döner. Anahtar genişletme (key schedule) ana anahtardan her tur için ayrı bir anahtar üretir. Bu tasarım 20+ yıldır pratik bir kırılma yaşamadı.

Anahtar terimler: S-box, SubBytes, ShiftRows, MixColumns, AddRoundKey, key schedule, $GF(2^8)$

► YAPAY ZEKAYA SOR

1. “AES-128'in bir turunu tek bir örnek blok üzerinde adım adım işle; her adımdan sonra matrisin hâlini göster.”
2. “AES'in S-box'ı nasıl üretildi ve neden rastgele bir tablo değil? Matematiksel gerekçesini açıkla.”
3. “AES neden 128 bit anahtarda 10 tur dönüyor? Tur sayısı düşürülürse hangi saldırılar mümkün hâle gelir?”

2.4 Çalışma Modları: ECB, CBC, CTR, GCM

Blok şifre tek başına yalnızca 16 baytlık bir bloğu şifreler; uzun veri için bir 'mod' gerekir. ECB her bloğu bağımsız şifreler ve aynı düz metin aynı şifreli metni verdiği için resimlerdeki desenleri sızdırır; asla kullanılmamalıdır. CBC her bloğu bir öncekiyle XOR'lar ve rastgele bir IV ister. CTR blok şifreyi akış şifresine dönüştürür, paralelleşir. GCM ise CTR'ye kimlik doğrulama etiketi (tag) ekler; hem gizlilik hem bütünlük sağlar (AEAD). Nonce'un asla tekrar kullanılmaması kritik bir kuraldır.

Anahtar terimler: ECB, CBC, CTR, GCM, IV, nonce, padding, AEAD, padding oracle

► YAPAY ZEKAYA SOR

1. “ECB modunun 'penguen görüntüsü' problemini teknik olarak açıkla ve neden desenlerin sızdığını göster.”
2. “CBC, CTR ve GCM modlarını şema mantığıyla anlat; her biri için 'hangi hata felaket olur' örneği ver.”
3. “Nonce tekrar kullanımının GCM ve CTR modlarında yol açtığı saldırıyı matematiksel olarak göster.”

2.5 Anahtar Türetme: Salt, KDF, Argon2

Kullanıcı parolası doğrudan anahtar olarak kullanılamaz çünkü entropisi düşüktür. Anahtar türetme fonksiyonları (KDF) parolayı yavaşça ve tuzlayarak (salt) anahtara dönüştürür. PBKDF2 tur sayısı, bcrypt ve scrypt bellek maliyetiyle, Argon2 ise hem bellek hem paralellik maliyetiyle saldırganı yavaşlatır. Salt, aynı parolaya sahip iki kullanıcının aynı çıktıyı üretmesini ve gökkuşağı tablolarını engeller. Blockchain cüzdanlarında seed phrase'den anahtar üretimi de bir KDF (PBKDF2) ile yapılır.

Anahtar terimler: KDF, salt, pepper, PBKDF2, bcrypt, scrypt, Argon2, rainbow table

► YAPAY ZEKAYA SOR

1. “Parola saklamada salt, pepper ve KDF kavramlarını ayrı ayrı anlat; birini atlarsam ne olur senaryosu kur.”
2. “PBKDF2, bcrypt, scrypt ve Argon2'yi maliyet parametreleri ve GPU/ASIC direnci açısından karşılaştır.”
3. “BIP-39 seed phrase'inden özel anahtarın nasıl türetildiğini PBKDF2 adımlarıyla açıkla.”

■ GÜNÜN UYGULAMASI

Python 'cryptography' kütüphanesiyle aynı görseli ECB ve CBC modunda şifrele, çıktıları karşılaştır. Ardından AES-GCM ile bir metni şifreleyip tag'i tek bit değiştirerek çözme denemesi yap ve hatayı gözlemle.

● KENDİNİ SINA

1. ECB modu neden hiçbir zaman önerilmez?
2. AEAD ne demektir ve GCM'yi CBC'den ayıran nedir?
3. Salt kullanılmazsa hangi saldırı kolaylaşır?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 3 · 5-6 saat

Asimetrik Kriptografi

İki anahtarın gücü: Diffie-Hellman, RSA, eliptik eğriler ve PKI

Günün hedefi: Açık anahtar kriptografisinin mantığını, RSA ve ECC'nin dayandığı zor problemleri ve blockchain adreslerinin nereden geldiğini kavrayacaksınız.

3.1 Açık Anahtar Fikri

1976'da Diffie ve Hellman şunu önerdi: iki anahtar üretelim; biri herkese açık olsun (public key), diğeri gizli kalsın (private key). Açık anahtarla şifrelenen yalnızca gizli anahtarla çözülür; gizli anahtarla imzalanan yalnızca açık anahtarla doğrulanır. Bu, binlerce yıllık anahtar dağıtım problemini çözdü. Güvenliği 'kapaklı tek yönlü fonksiyon' varsayımına dayanır: çarpanlara ayırma veya ayırık logaritma gibi tersi pratikte hesaplanamayan işlemler. Blockchain'de cüzdan adresi açık anahtardan türetilir, harcama yetkisi ise gizli anahtardadır.

Anahtar terimler: public key, private key, key pair, trapdoor function, asimetrik şifreleme

► YAPAY ZEKAYA SOR

1. “Açık anahtar kriptografisini posta kutusu ve kilitli kasa benzetmesiyle anlat, sonra benzetmenin nerede yetersiz kaldığını söyle.”
2. “Aynı anahtar çifti hem şifreleme hem imzalama için nasıl kullanılıyor? İki senaryoyu ayrı ayrı adım adım göster.”
3. “Asimetrik şifreleme neden büyük dosyaları şifrelemek için kullanılmaz? Hibrit şifrelemeyi (TLS örneğiyle) anlat.”

3.2 Diffie-Hellman Anahtar Değişimi

İki taraf, dinlenen bir kanal üzerinden konuşarak ortak bir gizli anahtarda anlaşabilir. Herkese açık p ve g değerleri seçilir; Alice gizli a ile $g^a \bmod p$, Bob gizli b ile $g^b \bmod p$ gönderir. Her biri gelen değeri kendi gizliyle üsler ve ikisi de $g^{(ab)} \bmod p$ değerine ulaşır. Dinleyici g^a ve g^b 'yi görse bile ayırık logaritma zorluğu nedeniyle g^{ab} 'yi hesaplayamaz. Ancak DH tek başına kimlik doğrulamaz; imzayla desteklenmezse MITM saldırısına açıktır. Modern TLS, her oturum için yeni anahtar üreten ECDHE'yi kullanır (ileri gizlilik).

Anahtar terimler: DH, ECDHE, ayırık logaritma, ortak sır, forward secrecy, MITM

► YAPAY ZEKAYA SOR

1. “Diffie-Hellman'ı küçük sayılarla ($p=23$, $g=5$) baştan sona hesaplayarak göster ve boya karıştırma benzetmesiyle destekle.”
2. “Diffie-Hellman'a yapılan ortadaki adam saldırısını canlandır ve sertifikaların bunu nasıl engellediğini açıkla.”
3. “İleri gizlilik (forward secrecy) nedir? Uzun ömürlü anahtar kullanımıyla ephemeral anahtar kullanımı karşılaştır.”

3.3 RSA: Üretim, Şifreleme, İmza

İki büyük asal p ve q seçilir, $n = p \cdot q$ hesaplanır. $\phi(n) = (p-1)(q-1)$ üzerinden bir açık üs e ve onun modüler tersi d bulunur. Şifreleme $c = m^e \bmod n$, çözme $m = c^d \bmod n$ şeklindedir. Güvenliği n 'yi çarpanlarına ayırmanın zorluğuna dayanır; 2048 bit bugünün standardıdır. Ham RSA ('textbook RSA') güvensizdir: OAEP dolgusu şifreleme için, PSS ise imza için zorunludur. RSA yavaş olduğu için genelde yalnızca simetrik anahtarı taşımak veya imza atmak için kullanılır.

Anahtar terimler: RSA, çarpanlara ayırma, e ve d , OAEP, PSS, textbook RSA, 2048-bit

► YAPAY ZEKAYA SOR

1. "RSA anahtar çiftini $p=61$, $q=53$ ile baştan sona üret; e ve d 'yi hesapla, bir mesajı şifreleyip çöz."
2. "Textbook RSA'nın hangi saldırılara açık olduğunu (deterministiklik, çarpımsallık, küçük e) örneklerle anlat."
3. "RSA-2048 kırmak için gereken hesaplama gücünü açıkla ve kuantum bilgisayarların Shor algoritmasıyla bunu nasıl değiştireceğini anlat."

3.4 Eliptik Eğri Kriptografisi (ECC)

Eliptik eğriler, $y^2 = x^3 + ax + b$ denklemiyle tanımlanan noktalar kümesidir ve bu noktalar üzerinde bir 'toplama' işlemi tanımlanabilir. Bir G noktasını k kez toplamak kolay, sonuçtan k 'yı bulmak (ECDLP) imkânsızdır. Aynı güvenlik için RSA'dan çok daha kısa anahtar gerekir: 256 bit ECC $\approx$ 3072 bit RSA. Bitcoin ve Ethereum secp256k1 eğrisini ve ECDSA imza algoritmasını kullanır; Ed25519 (EdDSA) ise daha modern ve hatalara dayanıklı bir alternatiftir. Cüzdan adresin, aslında bu eğri üzerindeki bir noktanın hash'idir.

Anahtar terimler: ECC, secp256k1, ECDSA, EdDSA, Ed25519, generator point, ECDLP

► YAPAY ZEKAYA SOR

1. "Eliptik eğri üzerinde nokta toplama ve skaler çarpma işlemlerini geometrik olarak anlat, küçük bir sonlu cisimde örnek yap."
2. "Bitcoin'de özel anahtardan açık anahtara, oradan adrese giden dönüşümü adım adım (secp256k1, SHA-256, RIPEMD-160, Base58Check) göster."
3. "ECDSA'da nonce (k) tekrar kullanılırsa özel anahtar nasıl ele geçirilir? PlayStation 3 vakasını da anlatarak matematiği göster."

3.5 PKI, Sertifikalar ve TLS

Açık anahtarın gerçekten o kişiye ait olduğunu nereden biliriz? Cevap: güvenilen bir üçüncü taraf onu imzalar. Sertifika Otoriteleri (CA) X.509 sertifikaları düzenler, tarayıcılar kök sertifikaları önceden tanır ve bir güven zinciri oluşur. TLS el sıkışmasında sunucu sertifikasını sunar, taraflar ECDHE ile oturum anahtarını türetir ve sonrası AES-GCM ile simetrik olarak şifrelenir. Blockchain ise bu merkezî güven modelini reddeder: kimse seni onaylamaz, anahtarın kendisi kimliğindir.

Anahtar terimler: PKI, CA, X.509, güven zinciri, kök sertifika, TLS handshake, CRL/OCSP

► YAPAY ZEKAYA SOR

1. “TLS 1.3 el sıkışmasını mesaj mesaj anlat ve her adımda hangi kriptografik aracın devrede olduğunu belirt.”
2. “PKI'daki merkezî güven modelinin zayıflıklarını (kötü niyetli CA, sertifika sahteciliği) gerçek vakalarla anlat.”
3. “Blockchain'in 'güvensiz güven' modeliyle PKI'nın merkezî güven modelini karşılaştır; hangisi hangi problemde üstün?”

■ GÜNÜN UYGULAMASI

OpenSSL ile RSA anahtar çifti üret, bir dosyayı imzala ve doğrula. Ardından Python ecdsa/coincurve kütüphanesiyle secp256k1 özel anahtarından bir Bitcoin adresi üret ve adım adım hash dönüşümlerini yazdır.

● KENDİNİ SINA

1. Diffie-Hellman tek başına neden kimlik doğrulamaz?
2. 256 bit ECC neden 3072 bit RSA ile eşdeğer kabul edilir?
3. Bir sertifikanın güven zinciri nasıl doğrulanır?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 4 · 4-5 saat

Hash Fonksiyonları, İmzalar ve Merkle Ağaçları

Blockchain'in yapıştırıcısı: özet fonksiyonları ve kanıtlanabilir bütünlük

Günün hedefi: Hash fonksiyonlarının özelliklerini, dijital imzanın işleyişini ve Merkle ağaçlarının neden blockchain'in kalbi olduğunu anlayacaksınız.

4.1 Kriptografik Hash Fonksiyonlarının Özellikleri

Hash fonksiyonu her uzunluktaki girdiyi sabit uzunlukta bir özete dönüştürür. Kriptografik olması için üç şart gerekir: ön görüntü direnci (özetten girdiyi bulamama), ikinci ön görüntü direnci (aynı özeti veren farklı girdi üretememe) ve çakışma direnci (herhangi iki farklı girdinin aynı özeti vermemesi). Ayrıca çığ etkisi vardır: girdinin tek biti değişirse çıktının yarısı değişir. Doğum günü paradoksu nedeniyle n bitlik bir hash'in çakışma direnci $n/2$ bittir; SHA-256'nın çakışma güvenliği 128 bittir.

Anahtar terimler: hash, digest, preimage, collision, çığ etkisi, doğum günü paradoksu, deterministik

► YAPAY ZEKAYA SOR

1. "Kriptografik hash'in üç direnç özelliğini birbirinden ayırt edebilmem için her biri farklı bir saldırı senaryosu kur."
2. "Doğum günü paradoksunu anlat ve SHA-256'da çakışma bulmanın neden 2^{128} işlem gerektirdiğini hesapla."
3. "Hash fonksiyonu ile şifreleme ve kodlama (Base64) arasındaki farkları karıştırılan noktalara vurgu yaparak açıkla."

4.2 SHA Ailesi ve Kırılan Algoritmalar

MD5 (1992) ve SHA-1 (1995) bugün çakışma saldırılarına yenik düşmüştür; 2017'de Google iki farklı PDF için aynı SHA-1 özetini üretti (SHAttered). SHA-2 ailesi (SHA-256, SHA-512) Merkle-Damgård yapısını kullanır ve hâlâ güvenlidir; Bitcoin SHA-256'yı iki kez uygular. SHA-3 (Keccak) ise tamamen farklı bir 'sünger' (sponge) yapısına dayanır ve uzunluk uzatma saldırısına karşı doğal olarak dayanıklıdır. Ethereum, standartlaşmadan önceki Keccak-256'yı kullanır; bu ince fark geliştiricileri sık şaşırtır.

Anahtar terimler: MD5, SHA-1, SHA-256, SHA-3, Keccak, Merkle-Damgård, sponge, length extension

► YAPAY ZEKAYA SOR

1. "SHA-256'nın iç işleyişini (mesaj bölme, genişletme, sıkıştırma turları) sadeleştirerek anlat."
2. "SHAttered saldırısını anlat: iki farklı PDF aynı SHA-1 özetini nasıl verdi ve pratik sonucu ne oldu?"
3. "Uzunluk uzatma (length extension) saldırısını örnekle göster ve SHA-3 ile HMAC'in bunu nasıl engellediğini açıkla."

4.3 HMAC ve Mesaj Doğrulama

Hash tek başına bütünlüğü kanıtlamaz; saldırgan hem mesajı hem özeti değiştirebilir. HMAC, gizli bir anahtarı hash'e belirli bir yapıda (iç ve dış pad) karıştırarak mesajın hem değişmediğini hem de anahtarı bilen biri tarafından üretildiğini kanıtlar. API isteklerinin imzalanması, webhook doğrulama ve JWT (HS256) hep HMAC kullanır. HMAC simetrik: doğrulayabilen aynı zamanda üretebilir; bu yüzden inkâr edilemezlik sağlamaz, onun için dijital imza gerekir.

Anahtar terimler: MAC, HMAC, ipad/opad, JWT, sabit zamanlı karşılaştırma, timing attack

► YAPAY ZEKAYA SOR

1. “HMAC'in neden basitçe hash(anahtar + mesaj) şeklinde tasarlanmadığını uzunluk uzatma saldırısıyla açıkla.”
2. “HMAC ile dijital imzayı 'kim doğrulayabilir, kim üretebilir, inkâr edilebilir mi' eksenlerinde karşılaştır.”
3. “MAC karşılaştırmasında sabit zamanlı (constant-time) fonksiyon kullanılmazsa oluşan timing saldırısını anlat.”

4.4 Dijital İmza

Dijital imzada mesajın özeti alınır ve bu özet gizli anahtarla imzalanır; herkes açık anahtarla doğrulayabilir. Bu, hem bütünlüğü hem kimliği hem de inkâr edilemezliği aynı anda sağlar. Blockchain'de her işlem, harcanacak fonun sahibi tarafından imzalanır; ağdaki her düğüm bu imzayı bağımsızca doğrular. Bu yüzden blockchain'de 'hesap' yoktur, 'imza yetkisi' vardır: özel anahtarı kaybeden kişi paranın sahipliğini de kaybeder, çünkü kimlik anahtardan ibarettir.

Anahtar terimler: dijital imza, ECDSA, imza doğrulama, non-repudiation, r-s değerleri, malleability

► YAPAY ZEKAYA SOR

1. “Bir blockchain işleminin imzalanma ve doğrulanma sürecini baştan sona, hangi verinin hash'lendiğini de belirterek anlat.”
2. “İslak imza, elektronik imza ve dijital imza kavramlarını hukuki ve teknik açıdan ayrıştır.”
3. “İmza esnekliği (transaction malleability) nedir ve Bitcoin'de SegWit bunu nasıl çözdü?”

4.5 Merkle Ağaçları ve Kanıtlar

Merkle ağacı, verileri ikiye bölerek tek bir kök özete (Merkle root) indirger. Bunun gücü şudur: bin işlemde birinin ağaçta olduğunu kanıtlamak için bin işlemi değil, yalnızca yaklaşık $\log_2(1000) \approx 10$ hash'i göstermek yeterlidir. Bu 'Merkle proof' sayesinde hafif istemciler (SPV cüzdanları) tüm zinciri indirmeden doğrulama yapabilir. Blok başlığında yalnızca Merkle kök bulunur; tek bir işlemin değişmesi kökü değiştirir ve sahtecilik anında ortaya çıkar. Ethereum daha gelişmiş bir yapı olan Merkle-Patricia Trie kullanır.

Anahtar terimler: Merkle root, Merkle proof, SPV, yaprak/düğüm, Patricia Trie, log(n) doğrulama

► YAPAY ZEKAYA SOR

1. “8 işlemden oluşan bir Merkle ağacı çiz gibi anlat ve 3. işlem için Merkle proof'un hangi hash'lerden oluştuğunu göster.”
2. “Merkle ağacı olmasaydı blockchain doğrulaması ne kadar pahalı olurdu? Karşılaştırmalı hesap yap.”
3. “Ethereum'un Merkle-Patricia Trie yapısını Bitcoin'in basit Merkle ağacından ayıran özellikleri açıkla.”

4.6 Proof of Work'ün Hash Temeli

Proof of Work aslında bir hash bulmacasıdır: blok başlığındaki nonce değeri değiştirilerek, SHA-256 çıktısı belirli bir hedeften küçük olana kadar denenir. Çıktı öngörülemez olduğu için tek yöntem kaba kuvvettir; bu da elektrik ve donanım maliyeti demektir. Bulunduğunda ise doğrulaması tek bir hash işlemiyle anında yapılır: 'bulması zor, doğrulaması kolay'. Fikrin kökeni Adam Back'in 1997'deki Hashcash spam önleme sistemidir. Zorluk (difficulty) her 2016 blokta yeniden ayarlanarak ortalama blok süresi 10 dakikada tutulur.

Anahtar terimler: PoW, nonce, difficulty, target, hashcash, hash rate, difficulty adjustment

► YAPAY ZEKAYA SOR

1. “Proof of Work'ü bir hash bulmacası olarak anlat ve 4 sıfırla başlayan bir hash bulmanın beklenen deneme sayısını hesapla.”
2. “Bitcoin'in zorluk ayarlama mekanizmasını formülüyle açıkla; ağ hash gücü iki katına çıkarsa ne olur?”
3. “Hashcash'ten Bitcoin'e giden fikri çizgiyi anlat: spam önleme nasıl para birimine dönüştü?”

■ GÜNÜN UYGULAMASI

Python'da bir metnin SHA-256'sını al, tek bir harfi değiştirip çıkış etkisini gözlemle. Ardından 4 işlemli basit bir Merkle ağacı sınıfı yaz ve Merkle proof üretip doğrula. Son olarak nonce artırarak '0000' ile başlayan hash arayan bir mini madenci yaz.

● KENDİNİ SINA

1. Çakışma direnci neden $n/2$ bittir?
2. HMAC ile dijital imza arasındaki temel fark nedir?
3. Merkle proof'un boyutu işlem sayısı ile nasıl değişir?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 5 · 5-6 saat

Blockchain Mimarisi ve Konsensüs

Dağıtık defterin anatomisi: bloklar, ağ, mutabakat ve cüzdanlar

Günün hedefi: Bir blockchain'in veri yapısını, ağ katmanını, konsensüs mekanizmalarını ve cüzdan mimarisini uçtan uca açıklayabileceksin.

5.1 Blockchain Nedir? Çift Harcama Problemi

Blockchain, merkezî bir otorite olmadan üzerinde anlaşılan, ekleme-yalnızca (append-only) bir dağıtık defterdir. Çözdüğü asıl problem şudur: dijital bir varlık kopyalanabilirken, aynı parayı iki kez harcamak nasıl engellenir? Bitcoin'in 2008'deki cevabı, işlemleri zaman sırasına sokan ve değiştirilmesi maliyetli olan bir zincir kurmaktır. Buradaki yenilik kriptografi değil (hepsi zaten vardı), kriptografiyi ekonomik teşviklerle birleştirmektir. Bizans Generalleri Problemi'nin pratik bir çözümü olarak görülür.

Anahtar terimler: dağıtık defter, çift harcama, append-only, Bizans Generalleri, izinsiz (permissionless) ağ

► YAPAY ZEKAYA SOR

1. “Çift harcama problemini anlat ve merkezî bankaların, blockchain'in bunu nasıl farklı yollarla çözdüğünü karşılaştır.”
2. “Bitcoin whitepaper'ını bölüm bölüm özetle ve her bölümün hangi teknik problemi çözdüğünü açıkla.”
3. “Bizans Generalleri Problemi'ni hikâyeleştir, ardından blockchain konsensüsüyle eşleştir.”

5.2 Blok Yapısı ve Zincirleme

Her blok bir başlık (header) ve işlem listesinden oluşur. Başlıkta önceki bloğun hash'i, Merkle kök, zaman damgası, zorluk hedefi ve nonce bulunur. 'Önceki bloğun hash'i' alanı zinciri oluşturur: bir bloktaki tek bir işlemi değiştirmek Merkle kökü, o da blok hash'ini, o da sonraki tüm blokları geçersiz kılar. Değişmezlik (immutability) böyle sağlanır; matematiksel bir imkânsızlık değil, ekonomik bir caydırıcılıktır. Genesis bloğu zincirin ilk halkasıdır ve önceki hash'i sıfırdır.

Anahtar terimler: blok başlığı, previous hash, genesis block, timestamp, blok yüksekliği, immutability

► YAPAY ZEKAYA SOR

1. “Bir Bitcoin blok başlığının 80 baytını alan alan aç ve her alanın ne işe yaradığını açıkla.”
2. “Zincirdeki 100. bloğu değiştirmek isteyen bir saldırganın yapması gerekenleri ve maliyetini adım adım hesapla.”
3. “Bitcoin ve Ethereum blok yapılarını karşılaştır: hangi alanlar farklı ve neden?”

5.3 P2P Ağ, Düğümler ve Mempool

Blockchain'in sunucusu yoktur; birbirine eşit düğümler (node) gossip protokolüyle veri yayar. Tam düğüm (full node) tüm kuralları bağımsız doğrular ve zinciri saklar; hafif düğüm yalnızca başlıkları tutar; arşiv düğümü tüm geçmiş durumu barındırır. Yayınlanan ama henüz bloğa girmemiş işlemler mempool'da bekler ve ücret teklifine göre önceliklendirilir. Düğümlerin bağımsız doğrulaması, madencilerin kural dışı blok üretmesini engelleyen asıl güç dengesidir.

Anahtar terimler: P2P, full node, light node, gossip, mempool, propagation, işlem ücreti

► YAPAY ZEKAYA SOR

1. “Bir işlemin cüzdandan çıkıp bloğa girmesine kadar geçen yolculuğu adım adım anlat (yayın, mempool, seçim, blok, onay).”
2. “Full node çalıştırmanın kullanıcıya ve ağa faydalarını, maliyetlerini ve 'don't trust, verify' ilkesini açıkla.”
3. “Mempool'da işlem ücreti nasıl belirlenir? RBF ve CPFP mekanizmalarını örnekle anlat.”

5.4 Konsensüs Mekanizmaları: PoW, PoS, BFT

Konsensüs, kimin bir sonraki bloğu ekleyeceğine ve hangi zincirin geçerli olduğuna karar verme kuralıdır. PoW'da hesaplama gücü yatırılır; enerji maliyeti güvenliği satın alır. PoS'ta ise doğrulayıcılar teminat (stake) kilitler ve kural ihlalinde bu teminat yakılır (slashing); Ethereum 2022'de bu modele geçti. BFT tabanlı sistemler (Tendermint gibi) daha az katılımcıyla anında kesinlik (finality) sunar. Her modelin bir bedeli vardır: PoW enerji, PoS sermaye yoğunlaşması, BFT ise ölçeklenme sınırı.

Anahtar terimler: PoW, PoS, slashing, validator, finality, fork choice, nothing-at-stake, DPoS, PBFT

► YAPAY ZEKAYA SOR

1. “PoW, PoS ve BFT konsensüslerini güvenlik varsayımı, enerji, merkeziyetsizlik ve finality açısından tablo hâlinde karşılaştır.”
2. “Ethereum'un Merge süreciyle PoS'a geçişini anlat: validator olmak ne gerektiriyor, slashing hangi durumlarda oluyor?”
3. “Olasılıksal kesinlik (Bitcoin) ile mutlak kesinlik (BFT) farkını bir borsa yatırma senaryosu üzerinden açıkla.”

5.5 UTXO ve Hesap Modeli, Cüzdanlar

Bitcoin UTXO modelini kullanır: bakiye diye bir alan yoktur, harcanmamış çıktıların toplamı bakiyendir; her işlem eski çıktıları tüketip yenilerini yaratır. Ethereum ise hesap modelini kullanır ve bakiye bir durum (state) değişkenidir; akıllı kontratlar için daha uygundur. Cüzdanlar ise anahtar yöneticisidir, para tutmaz. BIP-39 ile 12/24 kelimelik seed phrase üretilir, BIP-32 ile bundan hiyerarşik olarak sınırsız anahtar türetilir ve BIP-44 türetme yollarını standartlaştırır. Bu yüzden tek bir seed phrase tüm cüzdanı geri getirir.

Anahtar terimler: UTXO, hesap modeli, state, seed phrase, BIP-32/39/44, HD wallet, derivation path, nonce

► YAPAY ZEKAYA SOR

1. “UTXO ve hesap modelini aynı ödeme senaryosunda yan yana işleyerek farklarını, avantaj ve dezavantajlarını göster.”
2. “BIP-39 seed phrase'ten adrese giden türetme zincirini (entropi, mnemonic, seed, master key, child key, adres) adım adım anlat.”
3. “Sıcak cüzdan, soğuk cüzdan, donanım cüzdanı ve saklamalı (custodial) çözümleri risk analiziyle karşılaştır.”

5.6 Ölçeklenebilirlik Trilemması ve Çatallanmalar

Vitalik Buterin'in formüle ettiği trilemmaya göre bir blockchain merkeziyetsizlik, güvenlik ve ölçeklenebilirlikten aynı anda üçünü birden maksimize edemez; biri için diğerinden ödün verilir. Blok boyutunu büyütmek hızı artırır ama düğüm çalıştırmayı pahalılaştırıp merkezîleştirir. Kural değişiklikleri çatallanmalara yol açar: soft fork geriye uyumludur (SegWit), hard fork değildir ve zinciri ikiye ayırabilir (Bitcoin Cash, Ethereum Classic). Katman-2 çözümleri bu ödünleşimi zincir dışına taşıyarak aşmayı hedefler.

Anahtar terimler: trilemma, soft fork, hard fork, blok boyutu, TPS, Lightning, rollup

► YAPAY ZEKAYA SOR

1. “Blockchain trilemmasını üç köşesi ve ödünleşimleriyle anlat, her köşeyi önceliklendiren birer gerçek proje örneği ver.”
2. “Soft fork ve hard fork farkını SegWit ve Bitcoin Cash örnekleriyle teknik olarak açıkla.”
3. “Bitcoin blok boyutu savaşlarını (block size war) tarafların argümanlarıyla tarafsız biçimde özetle.”

■ GÜNÜN UYGULAMASI

Python'da 50 satırlık bir mini blockchain yaz: blok sınıfı, önceki hash bağlantısı, basit PoW ve zincir doğrulama fonksiyonu. Ardından bir bloğun verisini değiştirip zincirin nasıl geçersizleştirdiğini göster. Bir blok gezgininde (explorer) gerçek bir blok başlığını incele.

● KENDİNİ SINA

1. Blok başlığındaki hangi alan zinciri oluşturur?
2. PoS'ta güvenlik neye dayanır ve slashing ne yapar?
3. UTXO modelinde 'bakiye' nasıl hesaplanır?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 6 · 5-6 saat

Akıllı Kontratlar ve Ethereum

Programlanabilir para: EVM, Solidity, token standartları ve DeFi

Günün hedefi: EVM'nin nasıl çalıştığını, Solidity ile basit bir kontrat yazmayı ve token standartlarının ne işe yaradığını öğreneceksin.

6.1 Ethereum ve EVM

Ethereum, blockchain'i bir para defterinden 'dünya bilgisayarı'na dönüştürür. EVM (Ethereum Virtual Machine) yığın tabanlı, deterministik bir sanal makinedir; her düğüm aynı kodu çalıştırıp aynı sonuca ulaşır. Ethereum'da iki hesap türü vardır: dışsal hesaplar (EOA, özel anahtarla kontrol edilir) ve kontrat hesapları (kodla kontrol edilir). Akıllı kontrat, zincire yüklenmiş ve koşulları sağlandığında kendiliğinden çalışan koddur; 'kod kanundur' ifadesi buradan gelir ama bu, hataların da geri alınamaz olması demektir.

Anahtar terimler: EVM, bytecode, opcode, EOA, kontrat hesabı, world state, deterministik yürütme

► YAPAY ZEKAYA SOR

1. "EVM'nin çalışma mantığını yığın, bellek ve storage ayrımıyla anlat; basit bir opcode dizisini elle yürüt."
2. "Bitcoin Script ile EVM'yi Turing tamlığı, güvenlik ve esneklik açısından karşılaştır."
3. "'Kod kanundur' ilkesini DAO hack'i örneği üzerinden tartış: hard fork kararı doğru muydu, karşı argümanlar neler?"

6.2 Gas Mekanizması

Sonsuz döngü çalıştıran bir kontrat tüm ağı kilitleyebilirdi; gas bunu engeller. Her opcode'un bir gas maliyeti vardır ve kullanıcı işlem başlatırken gas limiti ile birim fiyatı belirler. Gas biterse işlem geri alınır (revert) ama harcanan gas iade edilmez. EIP-1559 ile ücret ikiye ayrıldı: yakılan taban ücret (base fee) ve doğrulayıcıya giden bahşiş (priority fee). Depolamaya yazmak (SSTORE) en pahalı işlemlerden biridir; bu yüzden akıllı kontrat optimizasyonu büyük ölçüde 'daha az yazma' sanatıdır.

Anahtar terimler: gas, gas limit, base fee, priority fee, EIP-1559, revert, out of gas, wei/gwei

► YAPAY ZEKAYA SOR

1. "Gas mekanizmasını sıfırdan anlat ve basit bir transfer ile bir NFT mint işleminin gas maliyetini karşılaştır."
2. "EIP-1559 öncesi ve sonrası ücret modelini karşılaştır; base fee yakımının ekonomik etkisi ne oldu?"
3. "Bir Solidity kontratında gas optimizasyonu için uygulanabilecek 10 tekniği örnek kodla göster."

6.3 Solidity Temelleri

Solidity, EVM için tasarlanmış, JavaScript'e benzeyen statik tipli bir dildir. Temel yapı taşları: durum değişkenleri (storage'da kalıcı), fonksiyonlar ve görünürlük belirteçleri (public, external, internal, private), modifier'lar (erişim kontrolü için), event'ler (zincir dışına log yayımlamak için) ve mapping veri yapısı. msg.sender çağıran adresi, msg.value gönderilen ETH miktarını verir. require ile koşul denetlenir ve sağlanmazsa işlem tamamen geri alınır; blockchain'de 'kısmen çalışmış işlem' diye bir şey yoktur.

Anahtar terimler: Solidity, storage/memory/calldata, modifier, event, mapping, msg.sender, require/revert, payable

► YAPAY ZEKAYA SOR

1. “Bana yorum satırlarıyla açıklanmış basit bir Solidity kontratı yaz (bağış kutusu) ve her satırın ne yaptığını anlat.”
2. “storage, memory ve calldata farkını gas maliyeti ve kullanım senaryolarıyla örnek kod üzerinde göster.”
3. “Solidity'de require, revert, assert ve custom error kullanımlarını ne zaman hangisi diye ayrıştırarak anlat.”

6.4 Token Standartları: ERC-20, ERC-721, ERC-1155

Standartlar, farklı cüzdan ve borsaların aynı arayüzü konuşmasını sağlar. ERC-20 değiştirilebilir (fungible) tokenlar içindir; transfer, approve ve transferFrom fonksiyonlarını tanımlar. approve/transferFrom ikilisi merkeziyetsiz borsaların temelidir ama sınırsız onay vermek büyük bir risktir. ERC-721 her birimi benzersiz olan NFT'leri, ERC-1155 ise aynı kontratta hem fungible hem non-fungible varlıkları verimli biçimde yönetir. Token aslında bir bakiye tablosudur; 'cüzdanında token durmaz', kontratın tablosunda adresine yazılı bir sayı vardır.

Anahtar terimler: ERC-20, ERC-721, ERC-1155, fungible, approve, allowance, mint/burn, metadata, tokenURI

► YAPAY ZEKAYA SOR

1. “ERC-20 standardının tüm fonksiyonlarını tek tek açıkla ve minimal bir ERC-20 kontratı yazıp satır satır yorumla.”
2. “approve/transferFrom akışını çiz gibi anlat; sınırsız approve vermenin riskini ve permit (EIP-2612) alternatifini açıkla.”
3. “ERC-721 ve ERC-1155 farklarını bir oyun içi eşya ekonomisi senaryosunda karşılaştıır.”

6.5 DeFi, DAO ve Oracle'lar

DeFi, finansal araçları kodla değiştirme denemesidir: AMM'ler (Uniswap gibi) emir defteri yerine $x*y=k$ formülüyle likidite havuzu üzerinden fiyat belirler; borç verme protokolleri aşırı teminatlandırma ve likidasyon mekanizmasıyla çalışır; stablecoin'ler değeri sabitlemeye çalışır. DAO'lar yönetimi token oylamasına devreder. Oracle'lar ise kritik bir boşluğu doldurur: blockchain kendi başına dış dünyayı (döviz kuru, hava durumu) göremez; Chainlink gibi oracle ağları bu veriyi zincire taşır ve manipölasyonları DeFi'nin en büyük saldırı yüzeylerinden biridir.

Anahtar terimler: AMM, likidite havuzu, impermanent loss, aşırı teminat, likidasyon, stablecoin, DAO, oracle, flash loan

► YAPAY ZEKAYA SOR

1. “Uniswap'in $x*y=k$ sabit çarpım formülünü sayısal bir örnekle işle ve slippage ile impermanent loss kavramlarını hesapla.”
2. “Aave/Compound tarzı bir borç verme protokolünün teminat oranı ve likidasyon mekanizmasını senaryolarla anlat.”
3. “Oracle manipülasyonu ve flash loan saldırılarını gerçek bir vaka üzerinden adım adım çöz.”

6.6 Geliştirme Araçları ve Deploy

Remix, tarayıcıda yazıp derleyip deploy edebildiğin en hızlı başlangıç ortamıdır. Hardhat (JavaScript) ve Foundry (Solidity ile test yazılan, çok hızlı) profesyonel geliştirme çerçeveleridir. Kontratlar önce yerel ağda, sonra Sepolia gibi test ağlarında denenir; test ETH'i musluklardan (faucet) alınır. Cüzdan bağlantısı için MetaMask, zincirle konuşmak için ethers.js veya viem, hazır güvenli kontratlar için OpenZeppelin kütüphanesi kullanılır. Deploy edilen kod değiştirilemez; bu yüzden proxy tabanlı yükseltilebilirlik desenleri vardır.

Anahtar terimler: Remix, Hardhat, Foundry, Sepolia, faucet, ethers.js, OpenZeppelin, ABI, proxy/upgradeable

► YAPAY ZEKAYA SOR

1. “Sıfırdan Hardhat projesi kurup bir ERC-20 tokenı yazmam, test etmem ve Sepolia'ya deploy etmem için adım adım rehber hazırla.”
2. “Foundry ile Solidity'de birim testi ve fuzz testi nasıl yazılır? Örnek test dosyasıyla göster.”
3. “ABI nedir ve bir frontend uygulaması ethers.js ile kontratla nasıl konuşur? Minimal örnek kodla anlat.”

■ GÜNÜN UYGULAMASI

Remix'te bir 'Bağış Kutusu' kontratı yaz: ETH kabul etsin, bağışçıları event ile kaydetsin, yalnızca sahibi çekebilsin. Sepolia test ağına deploy et ve MetaMask ile etkileşime gir. Ardından OpenZeppelin ile bir ERC-20 token üret.

● KENDİNİ SINA

1. Gas biterse kullanıcı harcadığını geri alır mı?
2. ERC-20'de approve neden riskli olabilir?
3. Oracle'a neden ihtiyaç duyulur?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

GÜN 7 · 5-6 saat

Güvenlik, İleri Konular ve Bitirme Projesi

Saldırıları, sıfır bilgi kanıtları, katman-2, post-kuantum ve proje

Günün hedefi: Yaygın zafiyetleri tanıyacak, ileri kriptografik konulara giriş yapacak ve öğrendiklerini bir bitirme projesinde birleştireceksin.

7.1 Akıllı Kontrat Zafiyetleri

En bilinen zafiyet reentrancy'dir: kontrat dış bir adrese ETH gönderirken kontrolü ona devreder, o adres tekrar aynı fonksiyonu çağırıp bakiye güncellenmeden defalarca çekim yapar; 2016'daki DAO hack'i böyle oldu. Çözüm 'checks-effects-interactions' deseni ve reentrancy guard'dır. Diğer yaygın hatalar: eksik erişim kontrolü, güvenilmeyen delegatecall, zincir üstü rastgelelik kullanımı (block.timestamp tahmin edilebilir), fiyat oracle'ına aşırı güven ve mantık hataları. Zincirde deploy edilen kod geri alınamaz, bu yüzden denetim (audit) ve test şarttır.

Anahtar terimler: reentrancy, checks-effects-interactions, access control, delegatecall, integer overflow, front-running, audit

► YAPAY ZEKAYA SOR

1. "Reentrancy saldırısını zafiyetli kontrat, saldırgan kontrat ve düzeltilmiş versiyonla kod üzerinde göster."
2. "SWC Registry / en yaygın 10 akıllı kontrat zafiyetini örnek kod ve önleme yöntemiyle listele."
3. "Aşağıdaki Solidity kontratını güvenlik denetçisi gibi incele ve bulduğun her zafiyeti önem derecesiyle raporla: [kontratını buraya yapıştır]"

7.2 Ağ Seviyesi Saldırıları

%51 saldırısında hash gücünün veya stake'in çoğunluğunu ele geçiren taraf zinciri yeniden yazarak çift harcama yapabilir; küçük zincirlerde bu gerçekten yaşanmıştır. Sybil saldırısında saldırgan çok sayıda sahte kimlik üretir; PoW ve PoS bunu maliyetli hâle getirerek engeller. Eclipse saldırısında bir düğümün tüm bağlantıları kuşatılarak ona sahte bir dünya gösterilir. MEV (maximal extractable value) ise sıralama yetkisinin kötüye kullanılmasıdır: sandwich saldırıları ve front-running kullanıcıya gizli bir vergi gibi yansır.

Anahtar terimler: %51 saldırısı, Sybil, eclipse, selfish mining, MEV, sandwich attack, front-running, reorg

► YAPAY ZEKAYA SOR

1. "%51 saldırısını adım adım anlat ve farklı zincirler için maliyetini karşılaştıran bir analiz yap."
2. "MEV nedir? Sandwich saldırısını bir Uniswap işlemi üzerinde sayılarla göster ve korunma yollarını anlat."
3. "Sybil ve eclipse saldırılarının farkını açıkla; her birine karşı ağ katmanında alınan önlemleri listele."

7.3 Anahtar Yönetimi ve Saklama

Kriptoda kayıpların çoğu algoritma kırılmasından değil, anahtar yönetimi hatalarından gelir. Seed phrase'i ekran görüntüsü olarak saklamak, buluta yüklemek veya bir siteye girmek en yaygın hatalardır. Donanım cüzdanları özel anahtarı cihazdan hiç çıkarmaz ve imzalamayı içeride yapar. Multisig (çoklu imza) M-of-N kuralıyla tek bir anahtarın kaybını felaket olmaktan çıkarır; Shamir Gizli Paylaşımı ise sırrı parçalara böler. Kurumsal düzeyde MPC tabanlı saklama çözümleri anahtarı hiç birleştirmeden imza üretir.

Anahtar terimler: seed phrase, donanım cüzdanı, multisig, M-of-N, Shamir Secret Sharing, MPC, cold storage, sosyal mühendislik

► YAPAY ZEKAYA SOR

1. “Bireysel bir kullanıcı için seed phrase saklama stratejisi tasarla: tehdit modeli, yedekleme, kalıtım planı dâhil.”
2. “Multisig ile Shamir Gizli Paylaşımı'nı çalışma mantığı ve kurtarma senaryoları açısından karşılaştır.”
3. “Kripto kullanıcılarını hedefleyen en yaygın 10 dolandırıcılık ve oltalama yöntemini teknik detayıyla anlat.”

7.4 Sıfır Bilgi Kanıtları (ZKP)

Sıfır bilgi kanıtı, bir iddianın doğru olduğunu, iddianın kendisi dışında hiçbir bilgi vermeden kanıtlamaktır: 'bu parolayı biliyorum' demeden parolayı bilmeyi kanıtlamak gibi. Üç şart aranır: tamlik, sağlamlık ve sıfır bilgi. zk-SNARK'lar kısa ve hızlı doğrulanır ama çoğu güvenilir kurulum (trusted setup) gerektirir; zk-STARK'lar kurulumla ihtiyaç duymaz ve kuantum dirençlidir ama kanıtları büyüktür. Kullanım alanları: gizlilik odaklı işlemler, kimlik doğrulama ve en önemlisi ZK-rollup'larda hesaplamayı zincir dışına taşıyıp tek kanıtla doğrulama.

Anahtar terimler: ZKP, zk-SNARK, zk-STARK, trusted setup, prover/verifier, Ali Baba mağarası, circuit

► YAPAY ZEKAYA SOR

1. “Sıfır bilgi kanıtını Ali Baba mağarası ve renk körü benzetmeleriyle anlat, sonra tamlik/sağlamlık/sıfır bilgi şartlarını eşleştir.”
2. “zk-SNARK ve zk-STARK'ı kanıt boyutu, doğrulama süresi, trusted setup ve kuantum direnci açısından karşılaştır.”
3. “ZK-rollup'ın optimistic rollup'tan farkını çekim süresi ve güvenlik varsayımları üzerinden açıkla.”

7.5 Katman-2 ve Ölçekleme

Katman-2 çözümleri işlemleri ana zincirin dışında yürütür ve güvenliği ana zincirden ödünç alır. Optimistic rollup'lar işlemleri geçerli varsayar ve itiraz süresi (fraud proof penceresi) tanır; bu yüzden çekimler günler sürebilir. ZK-rollup'lar ise her partiyi geçerlilik kanıtıyla gönderir, kesinlik hızlıdır. Bitcoin tarafında Lightning Network ödeme kanallarıyla anlık ve ucuz transfer sağlar. Sidechain'ler ise kendi güvenlik modeline sahiptir ve teknik olarak katman-2 sayılmaz; bu ayrım güvenlik açısından kritiktir.

Anahtar terimler: L2, optimistic rollup, ZK-rollup, fraud proof, validity proof, Lightning, sidechain, data availability

► YAPAY ZEKAYA SOR

1. “Katman-2 çözümlerini bir tabloda karşılaştır: güvenlik kaynağı, çekim süresi, maliyet, olgunluk.”
2. “Lightning Network ödeme kanallarının açılış, güncelleme ve kapanış aşamalarını teknik olarak anlat.”
3. “Veri erişilebilirliği (data availability) problemi nedir ve rollup güvenliği için neden kritiktir?”

7.6 Post-Kuantum Kriptografi

Yeterince güçlü bir kuantum bilgisayar, Shor algoritmasıyla çarpanlara ayırma ve ayrık logaritmayı verimli çözebilir; bu RSA, Diffie-Hellman ve ECC'yi (dolayısıyla bugünkü blockchain imzalarını) kırılabilir kılar. Grover algoritması ise simetrik şifreleri ve hash'leri yalnızca karekök kadar hızlandırır; AES-256 ve SHA-256 makul ölçüde dayanıklıdır. NIST 2024'te kafes tabanlı ML-KEM ve ML-DSA ile hash tabanlı SLH-DSA'yı standartlaştırdı. 'Şimdi topla, sonra çöz' (harvest now, decrypt later) riski nedeniyle geçiş bugünden başladı.

Anahtar terimler: Shor, Grover, kafes tabanlı kriptografi, ML-KEM (Kyber), ML-DSA (Dilithium), SLH-DSA, kriptografik çeviklik

► YAPAY ZEKAYA SOR

1. “Shor ve Grover algoritmalarının kriptografiye etkisini ayrı ayrı, hangi algoritmayı ne kadar zayıflattığını belirterek anlat.”
2. “Bitcoin ve Ethereum kuantum bilgisayarlara karşı hangi noktalarda kırılabilir? Geçiş için önerilen yolları tartış.”
3. “NIST post-kuantum standartlarını (ML-KEM, ML-DSA, SLH-DSA) çalışma mantığı ve kullanım alanlarıyla özetle.”

7.7 Regülasyon, Etik ve Bitirme Projesi

Teknik bilgi tek başına yeterli değildir: KYC/AML yükümlülükleri, Avrupa'daki MiCA düzenlemesi, vergilendirme ve gizlilik hakkı tartışmaları alanın ayrılmaz parçasıdır. Zincir analizi firmaları işlemleri kümeleyerek sahte anonimlik yanılgısını kırar; blockchain anonim değil, takma adlıdır (pseudonymous). Etik tarafta enerji tüketimi, spekülasyon zarar ve dolandırıcılık ekosistemi ciddi eleştirilerdir. Bugünün asıl işi ise bitirme projesi: yedi günün bilgisini uçtan uca çalışan tek bir yapıda birleştirmek.

Anahtar terimler: KYC/AML, MiCA, pseudonymity, zincir analizi, gizlilik, enerji tartışması, uyum (compliance)

► YAPAY ZEKAYA SOR

1. “Blockchain'de anonimlik ile takma adlılık farkını açıkla ve zincir analizi tekniklerinin nasıl çalıştığını anlat.”
2. “Kripto varlık düzenlemelerini (MiCA, KYC/AML yaklaşımları) merkeziyetsizlik ilkesiyle çatıştığı noktalar üzerinden tartış.”
3. “7 günde öğrendiğim konuları birleştiren, benim seviyeme uygun 3 farklı bitirme projesi öner; her biri için adım adım yol haritası, gerekli araçlar ve değerlendirme kriterleri ver.”

■ GÜNÜN UYGULAMASI

Bitirme projesi seçenekleri: (1) Kendi mini blockchain'ini imzalı işlemler, mempool ve PoW ile Python'da yaz. (2) Zafiyetli bir kontrat yaz, saldırıyı gerçekleştir, sonra düzelt ve denetim raporu hazırla. (3) ERC-20 token + basit bir dApp arayüzü yapıp testnet'e deploy et.

● KENDİNİ SINA

1. Reentrancy'yi önleyen tasarım deseni nedir?
2. ZK-rollup ile optimistic rollup arasındaki temel güvenlik farkı nedir?
3. Kuantum bilgisayarlar SHA-256'yı neden RSA kadar tehdit etmiyor?

İpucu: Cevaplarını yaz, sonra yapay zekaya “bu cevapları bir sınav görevlisi gibi puanla ve eksiklerimi söyle” de.

Ek: Daha İyi Prompt Yazma Rehberi

Aynı soru, farklı sorulduğunda çok farklı cevaplar üretir. Aşağıdaki on teknik, bu programdaki promptları kendi ihtiyacına göre uyarlayan içindir.

01	Rol ver	Yapay zekaya bir kimlik ve seviye ata. 'Sen 20 yıllık bir kriptografi eğitmenisin; bana lise düzeyinde matematik bilen bir mühendis gibi anlat.' Rol, cevabın derinliğini ve dilini doğrudan belirler.
02	Seviyeni söyle	'Bu konuyu hiç bilmiyorum' ile 'temel matematiğim var' arasındaki fark, gelen cevabın işine yarayıp yaramamasını belirler. Bildiklerini de yaz ki tekrar anlatmasın.
03	Çıktı formatını iste	Tablo, adım adım liste, sözde kod, karşılaştırma, 5 soruluk test... Format istemezsen düz paragraf gelir ve öğrenmesi zorlaşır.
04	Örnek üzerinden çalıştır	'Anlat' yerine 'küçük sayılarla baştan sona hesapla' de. Kriptografide bir kavramı gerçekten anlamamanın en hızlı yolu, elle bir kez çözmektir.
05	Ters yönde sor	'Bu nerede kırılır?', 'Hangi hatayı yaparsam felaket olur?', 'Bu yaklaşımın karşı argümanı ne?' Zafiyet üzerinden öğrenmek kalıcıdır.
06	Sokratik mod kur	'Bana doğrudan cevap verme; soru sorarak yönlendir ve yanlış yaptığımda ipucu ver.' Bu, pasif okumayı aktif düşünmeye çevirir.
07	Kendini test ettir	Her konu sonunda 'bana bu konudan 5 soruluk zor bir test yap ve cevaplarımı gerekçeleriyle değerlendir' de. Sinanmayan bilgi unutulur.
08	Kaynak ve doğrulama iste	'Bu bilginin doğruluğunu nasıl kontrol ederim, hangi standart veya makaleye bakmalıyım?' Yapay zeka hata yapabilir; özellikle sayısal detayları ve güncel verileri doğrula.
09	Analoji ve sınırını iste	'Bunu bir benzetmeyle anlat, sonra benzetmenin nerede yanıltıcı olduğunu söyle.' Eksik benzetmeler en sinsi yanlış anlamaların kaynağıdır.
10	Bağlamı taşı	Aynı sohbette ilerle ve 'dün öğrendiğim X ile bugünkü Y arasındaki bağlantıyı kur' de. Kriptografi ve blockchain katman katman inşa edilir.

Hazır Kalıp: Her Konu İçin Kullanabileceğin Şablon

Sen deneyimli bir kriptografi ve blockchain eğitmenisin. Ben [seviyeni yaz: örn. temel matematiği olan bir yazılım öğrencisiyim] ve şu an [konu adı] konusunu çalışıyorum. Bu konuyu bana şu sırayla anlat: (1) hangi problemi çözmek için doğdu, (2) sezgisel bir benzetme ve benzetmenin sınırı, (3) teknik işleyiş adım adım, (4) küçük sayılarla elle çözülmüş bir örnek, (5) pratikte en sık yapılan üç hata ve sonuçları, (6) blockchain'de nerede kullanılıyor. Sonunda bana bu konudan 5 soruluk bir test yap ve cevaplarımı bekle.

Program Sonrası: Nereye Devam Etmeli?

Derinleşmek istiyorsan	Katz & Lindell — Introduction to Modern Cryptography; Cryptopals challenges; Bitcoin ve Ethereum whitepaper'ları.
-------------------------------	---

Geliştirici olmak istiyorsan	Solidity dokümantasyonu, Foundry Book, OpenZeppelin kontratlarını satır satır okumak, Ethernaut ve Damn Vulnerable DeFi oyunları.
Güvenlik tarafına gitmek istiyorsan	Akıllı kontrat denetim raporlarını okumak, SWC Registry, Immunefi bug bounty programları, CTF yarışmaları.
Araştırma tarafına gitmek istiyorsan	ZK devreleri (Circom, Noir), konsensüs literatürü, post-kuantum standart dokümanları.